

Project OMNISCIENT

Context

In the evolution of autonomous AI, we are moving beyond simple chatbots to deploy agentic detectives tasked with solving complex cases by inspecting crime scenes and independently interrogating suspects. However, a major challenge in deploying such systems is that agentic reasoning often lacks the explainability and transparency required for public trust. For citizens to confidently trust an AI investigator, the system cannot function as a black-box oracle that simply reads a prompt and spits out a random verdict. Its entire investigative process including its reasoning, choices and assessments must be logical, auditable and sound (bonus points if mathematically sound or statistically sound).

Data

Each individual murder mystery includes a description of the victim, detailed metadata regarding the crime scene, and profiles for up to N suspects.

When processing dataset entries for your detective system, restrict your usage to the time, location, and victim fields; for suspect profiles, utilize only the name, introduction, relationship, and suspicion fields.

In this environment, the suspects are represented by LLM agents. Each agent is pre-loaded with a specific story-including their timeline, motive, and alibi-and is programmed to stay in character

throughout the interrogation. These agents also follow specific behavioral guidelines; for instance, a suspect might be instructed to protect the murderer or to react with heightened emotion if your detective becomes hostile.

To help you build and refine your system, you must separate the data into training and validation datasets. You should use the training set for initial development and the validation set to measure your system's accuracy and token efficiency before final submission.

This task to make an auditable agentic detective system has been divided into 3 parts (or 3 components if you will).

Part 1: The Interrogator Agent [20 points]

This is the agent that frames questions and collects testimonies from the witnesses

You need to decide in what format you'll process and store the collected testimonies, for example, propositional logic.

Implement the above agent using a local LLM and fine tune it to output the chosen format for the collected testimonies

You also need to decide how independent this module will be from the reasoning engine (the next part of the task), i.e will it only ask the questions the reasoning engine tells it to or maybe it will follow up with a question of its own or maybe ask its own questions if the situation demands it.

For this you'd need to define a proper communication protocol between the reasoning engine and the interrogator module.

Deliverables

1. An Interrogator agent class in a file of its own

2. Documentation for the source code and choices of the format of the stored testimonies

BONUS [20 points]

Implement dynamic persona integration

Instead of simply asking questions, your detective should adopt specific interrogation personas (e.g., sympathetic, hostile, or analytical)

This should be done without simply appending instructional text like "You are now a bad cop" to the system prompt as such an approach does not remain stable over long contexts.

This is useful as suspects will respond differently to different types of interrogation personas i.e maybe one suspect does not respond at all to intimidation tactics but rather to friendly interactions.

A complete documentation is necessary for the taken approach to the bonus task.

Part 2: The Investigative Loop [40 points]

This component which acts as the brain of the agent, serving as the central reasoning engine for the entire case should be systematic, principled and formalised

As new testimonies come in, it should have a mechanism to identify contradictions in testimonies and you also need to implement logic on how to proceed given such a discrepancy is recognised. You need to decide the mechanism for deciding when and what to ask which suspect.

There must also be terminal logic to conclude when the evidence is sufficient to identify the murderer or maybe terminate the case if found to be too confusing.

All the above mechanisms can be rule based or statistical based or intuition based or a mixture of two or more of the above but in the end the explanation for each of the mechanisms should be satisfactory enough for the general public.

Purely intuition based mechanisms will have less evaluation points as they hamper explainability.

Deliverables

We will provide a baseline Python script named `main.py` which will also serve as the entry point to your system.

An entry point is the specific file that serves as the starting gate for the entire application. Your task is to build your detective's logic and integrate it so that it is triggered through this script.

It handles the loading of crime scenes and the initialisation of suspect agents.

While you are expected to build your own modules, classes, and logic in separate files, the final execution must flow through this provided script.

So the final deliverables are

1. A complete agentic system whose entrypoint must be the provided main.py
2. You must include a Markdown file (README.md) that serves as a technical manual for your system. It should contain the following:

System Architecture

Provide a clear breakdown of how your agent is structured and explain how the different components interact with each other to solve a case.

Methodology and Design Choices

For each major component, provide a technical justification for your approach. Explain why you chose specific strategies, such as the specific reasoning framework used.

Performance Metrics

Report the results of your system when run against the validation dataset. This must include your accuracy rate and the total computational cost measured in tokens spent, time taken, etc.

Setup and Execution Guide

List all necessary dependencies and provide clear instructions on how to set up the environment and execute the main.py entry point.

Part 3: Case Report Generation [20 points]

The system must output a comprehensive final report for every case. This report should include

A chronological reconstruction of what actually happened during the crime.

A step-by-step breakdown of your detective's reasoning. Why did it ask a specific question? Why did it suspect a specific person?

If any sort of error occurs during an investigation, the details of where, when and why the error occurred must be present in the case report albeit in a human readable format.

Deliverables

1. The source code for the case report generation
2. Documentation for the approach for extensive reporting as well as the computational cost metrics.

Starting Point

You can find the data and the boilerplate main.py script here:

https://drive.google.com/drive/folders/1fd5y1edorA5-OZ37Fo3Nm6NfF_sM8mTX?usp=sharing

Evaluation Criteria

These are general aspects of the system that will be used to evaluate your submissions across the 3 parts the task has been divided into.

Investigative Accuracy

The most critical metric is the system's ability to correctly identify the murderer and their motive.

Logical Soundness and Transparency

Higher scores will be awarded to systems that provide a clear, auditable path to their conclusion. A detective that correctly identifies a suspect through sound evidence and logical deduction is superior to one that arrives at a verdict through opaque "black-box" reasoning.

Computational Efficiency

Your system will be evaluated on its resource management. We are looking for "intelligent" reasoning that achieves high accuracy with minimal token or compute consumption.

Behavioural Stability

The interrogator agent must maintain its assigned persona throughout the interrogation of a suspect. Systems that lose their "character" or contradict their own previous findings during long contexts will be penalised.

Code Quality and Documentation

Your code must be clean, modular, and easy for another engineer to interpret.

Implement standardized logging and exception handling.

Also we know you'll be using LLMs but at the very least we expect that you don't leave any emojis or unnecessary comments in the code.

Also use comments extensively and use a standard format for the comments.

Final Case Report Quality

The generated case report must be coherent, chronologically accurate, and provide a convincing justification for the investigative steps taken.